Deep learning methods in population genomics and phylogeography

Manolo Fernandez Perez
Real Jardín Botánico (CSIC) - Spain

@ManoloLearning
manolofernandez@rjb.csic.es
sites.google.com/site/manolofperez

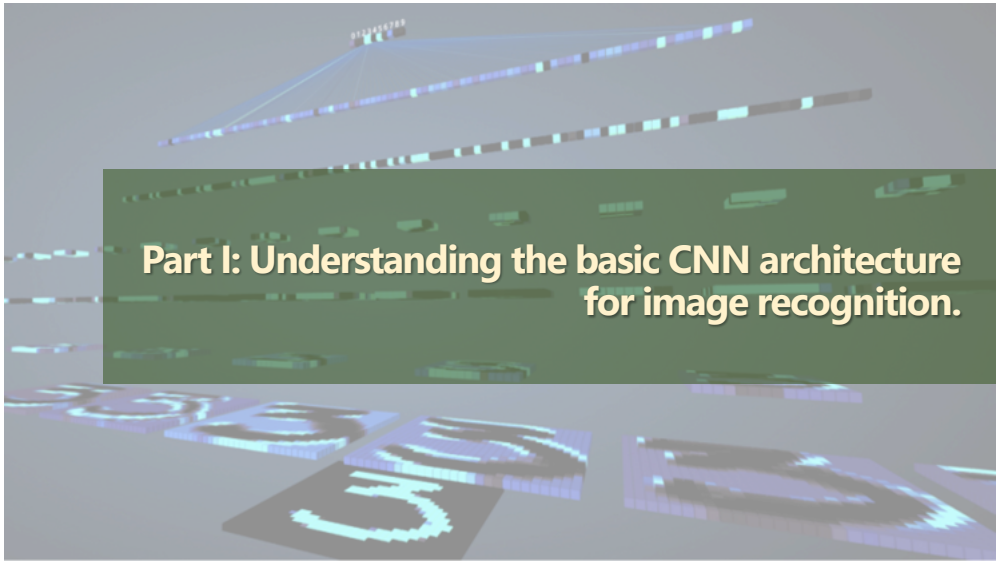
Goals

- Conceive and simulate genetic data under competing demographic scenarios
- Understand deep learning background and how a CNN works
- How to use deep learning to compare demographic scenarios

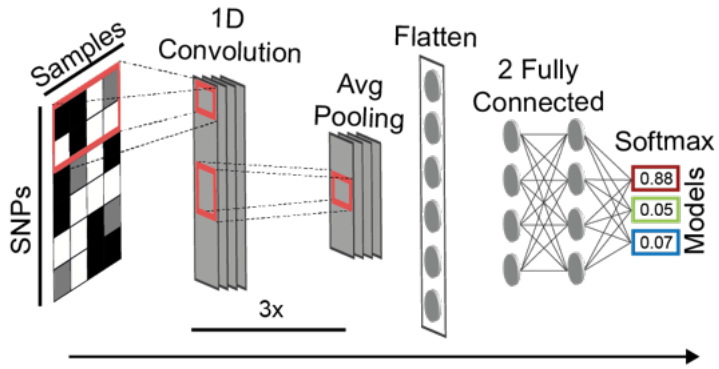
Program

Program

- *Part I: Understanding the basic CNN architecture for image recognition.*
- *Part II: Practical on Image classification with CNN.*
- *Part III: - The building blocks of a CNN script.*
- *Practical: Comparing demographic scenarios with deep learning.*

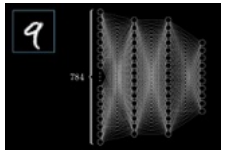


CNN architecture



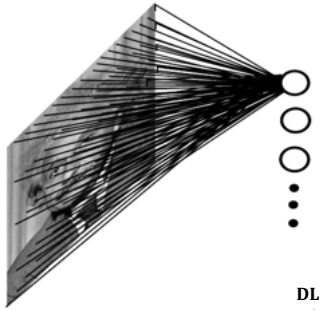
Kirschner et al. (2022) *Nat Comm*

Convolutional Neural Networks (CNNs)

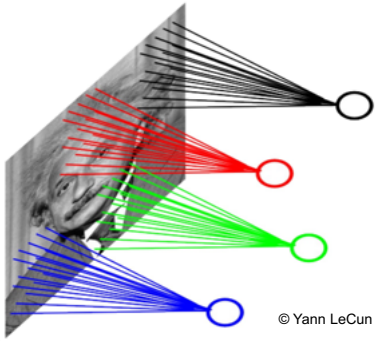


MLP
Fully connected

Convnet
locally connected
→ smaller number of parameters to learn



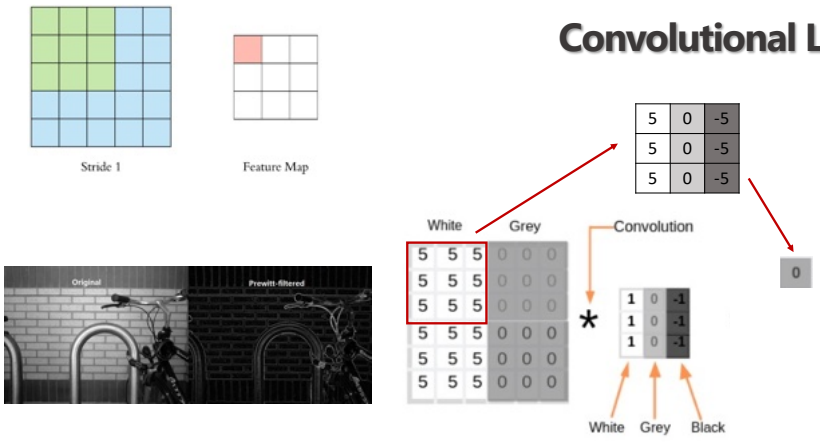
DL witchcraft



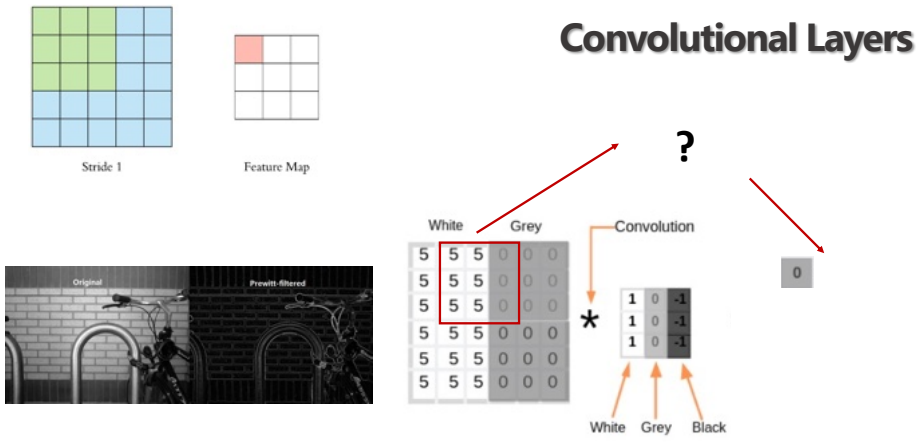
© Yann LeCun

Convolutional Layers

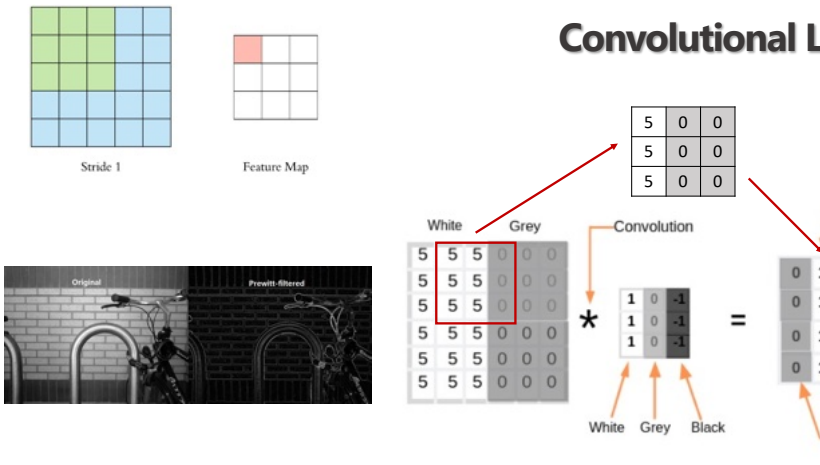
https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529



https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

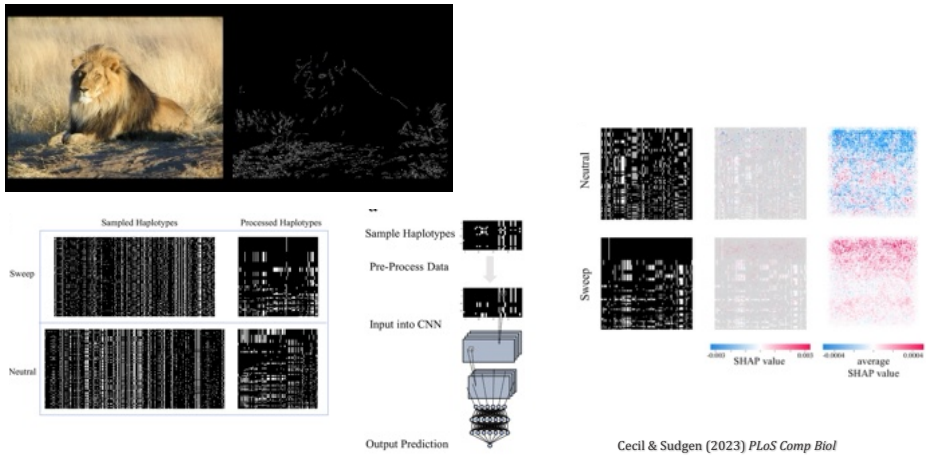


https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529



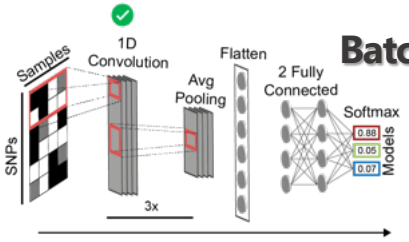
https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Neural Network for Genomic data

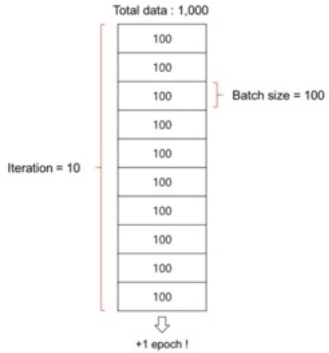


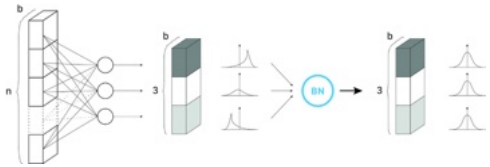
Cecil & Sudgen (2023) *PLoS Comp Biol*

Batches and Batch Normalization



Kirschner et al. (2022) *Nat Comm*

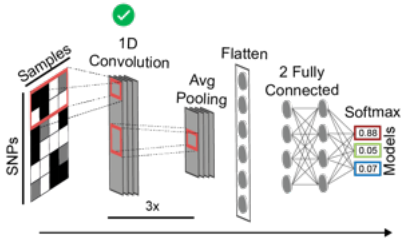




https://towardsdatascience.com/batch-normalization-in-3-levels-of-understanding-14c2da90a338

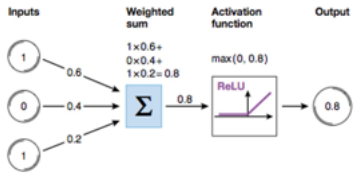
https://jerryan.medium.com/batch-size-a15958708a6

Activation Function

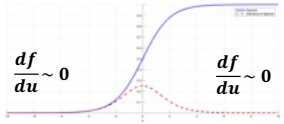


Kirschner et al. (2022) *Nat Comm*

DL witchcraft

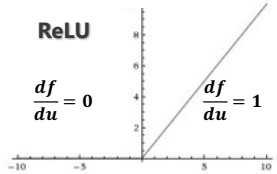


Sigmoid – Vanishing gradients



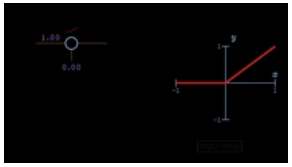
https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484

ReLU

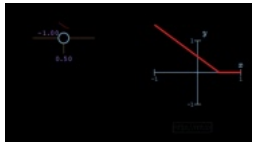


https://www.kaggle.com/code/dansbecker/rectified-linear-units-relu-in-deep-learning/notebook

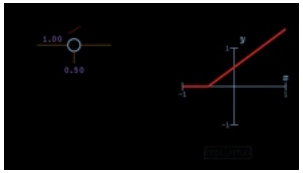
CNN Script – ReLU



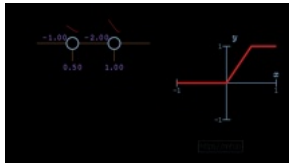
Weight = 1
Bias = 0



Weight = -1
Bias = 0.5



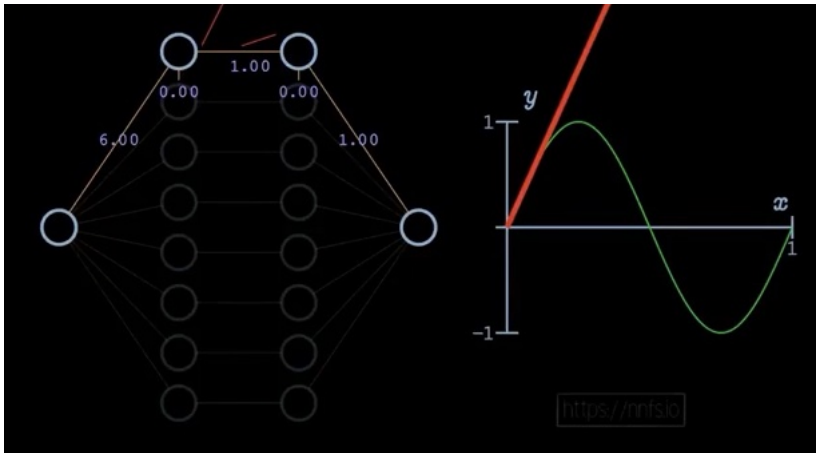
Weight = 1
Bias = 0.5



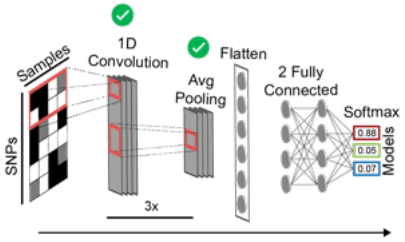
2 neurons

https://nnfs.io/mvp/

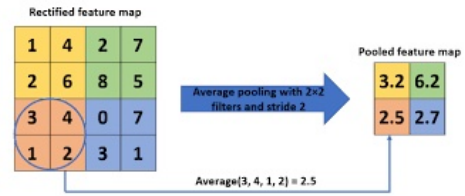
CNN Script – ReLU



https://nnfs.io/mvp/

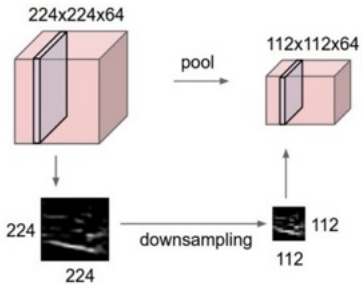


Kirschner et al. (2022) *Nat Comm*



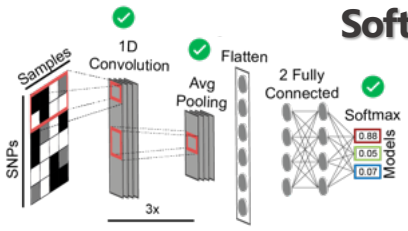
Gholamalinezhad & Khosravi (2020) *arXiv*

Pooling

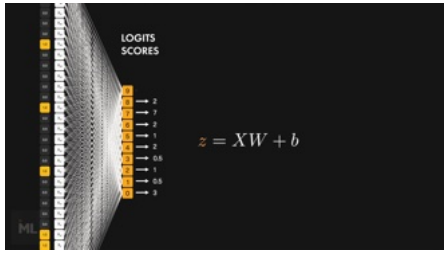


https://leonardoaraujosantos.gitbook.io/artificial-inteligence/machine_learning/deep_learning/pooling_layer

Softmax layer for Classification



Kirschner et al. (2022) *Nat Comm*



Libraries for deep Learning:
Keras, Tensorflow, pyTorch

K TensorFlow PyTorch

Hyperparameters

Table 2. Central parameters of a neural network and recommended settings.

Name	Range	Default value
Learning rate	0.1, 0.01, 0.001, 0.0001	0.01
Batch size	64, 128, 256	128
Momentum rate	0.8, 0.9, 0.95	0.9
Weight initialization	Normal, Uniform, Glorot uniform	Glorot uniform
Per-parameter adaptive learning rate methods	RMSprop, Adagrad, Adadelta, Adam	Adam
Batch normalization	Yes, no	Yes
Learning rate decay	None, linear, exponential	Linear (rate 0.5)
Activation function	Sigmoid, Tanh, ReLU, Softmax	ReLU
Dropout rate	0.1, 0.25, 0.5, 0.75	0.5
L1, L2 regularization	0, 0.01, 0.001	

Some of the best hyperparameter optimization libraries are:

1. Scikit-learn
2. Scikit-Optimize
3. Optuna
4. Hyperopt
5. Ray.tune
6. Talos
7. BayesianOptimization
8. Metric Optimization Engine (MOE)
9. Spearmint
10. GPyOpt
11. SigOpt
12. Fabolas

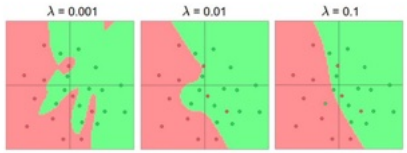
https://neptune.ai/blog/hyperparameter-tuning-in-python-complete-guide#hyperopt

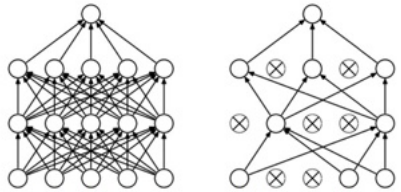
Table 2. Central parameters of a neural network and recommended settings.

Name	Range	Default value
Learning rate	0.1, 0.01, 0.001, 0.0001	0.01
Batch size	64, 128, 256	128
Momentum rate	0.8, 0.9, 0.95	0.9
Weight initialization	Normal, Uniform, Glorot uniform	Glorot uniform
Per-parameter adaptive learning rate methods	RMSprop, Adagrad, Adadelta, Adam	Adam
Batch normalization	Yes, no	Yes
Learning rate decay	None, linear, exponential	Linear (rate 0.5)
Activation function	Sigmoid, Tanh, ReLU, Softmax	ReLU
Dropout rate	0.1, 0.25, 0.5, 0.75	0.5
L1, L2 regularization	0, 0.01, 0.001	

Hyperparameters

Regularization





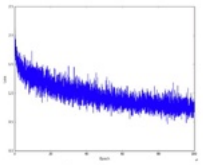
(a) Standard Neural Net (b) After applying dropout.

Hyperparameters

Table 2. Central parameters of a neural network and recommended settings.

Name	Range	Default value
Learning rate	0.1, 0.01, 0.001, 0.0001	0.01
Batch size	64, 128, 256	128
Momentum rate	0.8, 0.9, 0.95	0.9
Weight initialization	Normal, Uniform, Glorot uniform	Glorot uniform
Per-parameter adaptive learning rate methods	RMSprop, Adagrad, Adadelta, Adam	Adam
Batch normalization	Yes, no	Yes
Learning rate decay	None, linear, exponential	Linear (rate 0.5)
Activation function	Sigmoid, Tanh, ReLU, Softmax	ReLU
Dropout rate	0.1, 0.25, 0.5, 0.75	0.5
L1, L2 regularization	0, 0.01, 0.001	

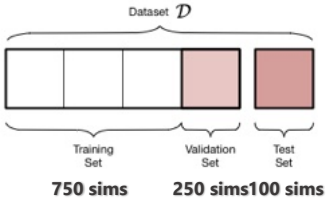
effects of different learning rates

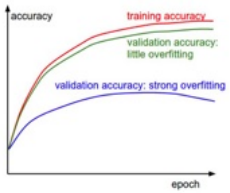


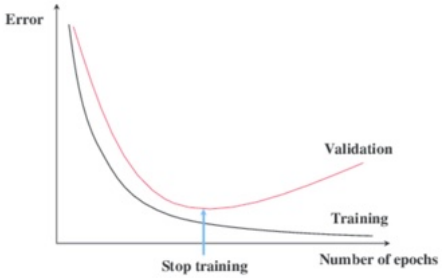
loss decay

Early Stop to avoid overfitting

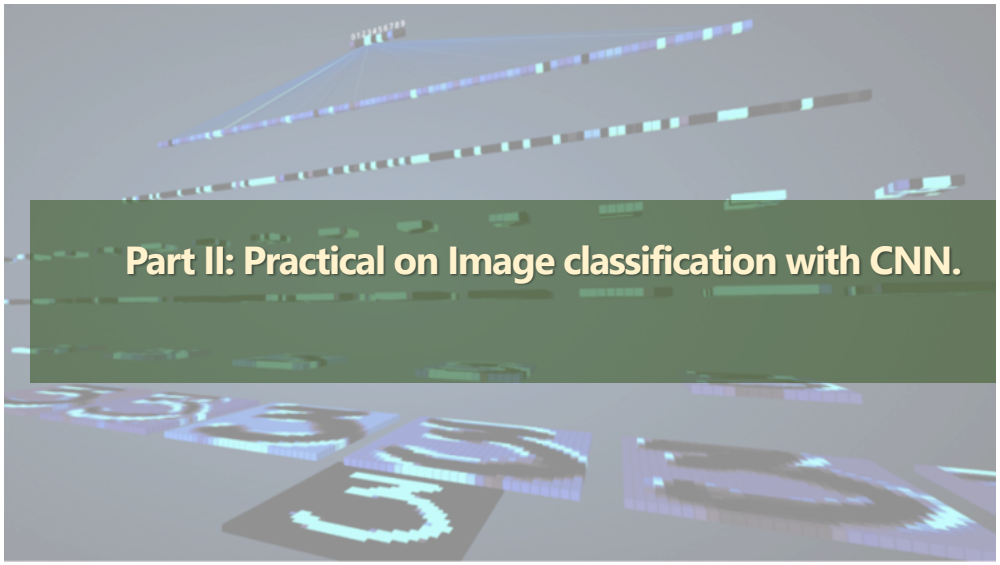
https://se-education.org/learningresources/contents/ai/ml.html







https://towardsdatascience.com/a-practical-introduction-to-early-stopping-in-machine-learning-550ac88bc8fd

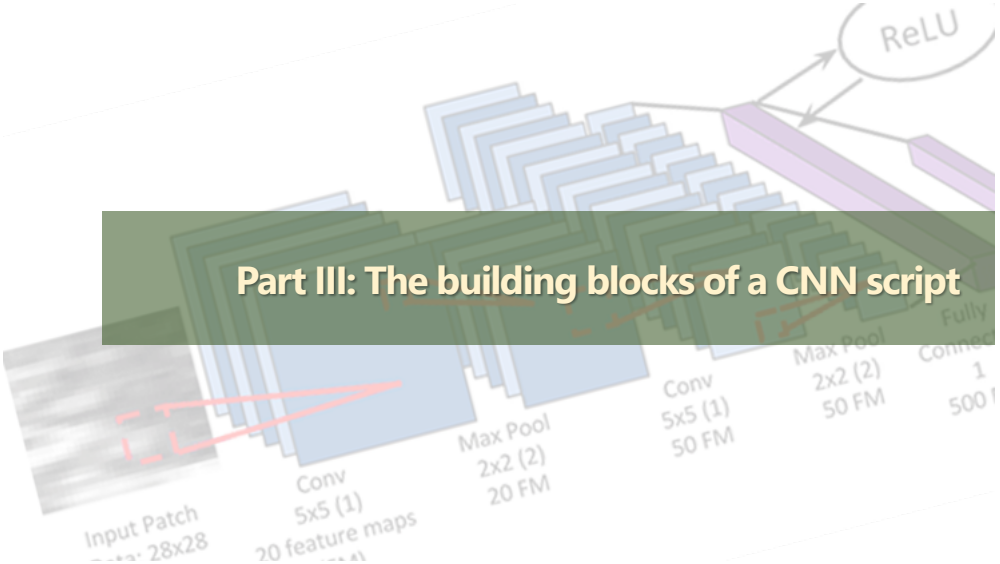


- **Practical Exercise 1:**
- Access https://poloclub.github.io/cnn-explainer/
- Scroll down to the Understanding hyperparameters section. Try to understand what is kernel size, padding, and stride.
- Use the graphical interface to try to better understand how convolutions work and what features are being extracted from the images.
- You can also upload the alignment images (Model1 and Model2.png) from different scenarios. How does the network's prediction for the alignments look like?

Goals

- Conceive and simulate genetic data under competing demographic scenarios
- Understand deep learning background and how a CNN works
- How to use deep learning to compare demographic scenarios

DL witchcraft

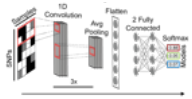


Part III: The building blocks of a CNN script

CNN Script

required python modules.

Define the CNN architecture.



Load and process the training data.

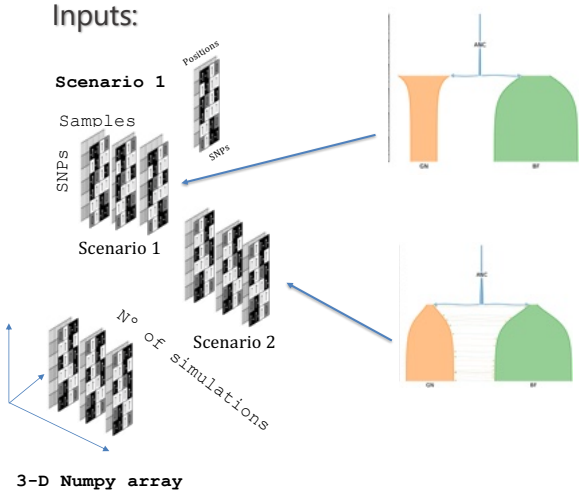
Train the network.

Load the test data and perform cross-validation.

Predict the most likely model for the empirical data

CNN Script

Inputs:



Parameters

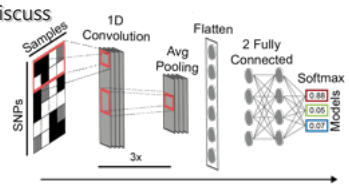
	Theta	T1	T2	T3	Ne
Sim1					
Sim2					
Sim3					
Sim4					

- **Practical Exercise 2:**

- Go through Section 1 of the notebook: https://drive.google.com/file/d/1vh5_8x4en673OvYu2H_qH-4DrWImkllc/view?usp=sharing

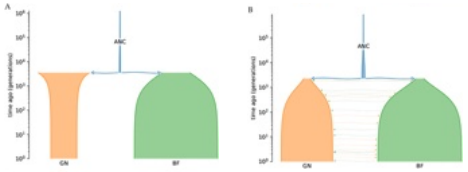
- We will try to recognize all the elements of the network. Do you remember the function of each of those elements? Remember that you can add annotations to the code using # and add information that might help you when you get back to the script in the future.

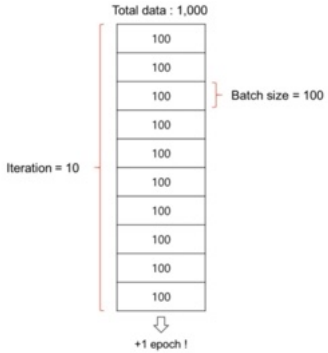
- Now run all the cells until you reach the end of section 2. Your network will be training, so now we will have some time to discuss and do a quick review on the CNN elements.



CNN Script

```
# Define parameters for the CNN run.
batch_size = 64
### how much interations to train the network
epochs = 100

###n of models
num_classes = 2
```

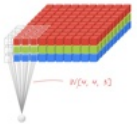




https://jerryan.medium.com/batch-size-a15958708a6

Kirschner et al. (2022) *Nat Comm*

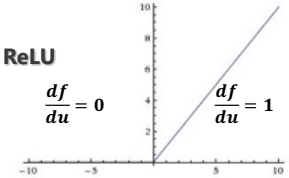
```
# Define the CNN architecture.
def create_cnn(xtrain):
    # Input shape: (250 SNPs, 280 Haplotypes, 2 Channels)input_shape = (xtrain.shape
    input_shape = (xtrain.shape[1], xtrain.shape[2], xtrain.shape[3])
    inputs = Input(shape=input_shape)

    # For 1D Conv: Reshape (250, 280, 2) -> (250, 560)
    # This combines Haplotypes and Channels into one "feature" dimension for Conv1D
    x = Reshape((xtrain.shape[1], xtrain.shape[2] * xtrain.shape[3]))(inputs)

    # 2. Convolutional Layers
    x = Conv1D(256, kernel_size=2, activation='relu')(x)
    x = AveragePooling1D(pool_size=2)(x)
    x = BatchNormalization()(x)
```



CNN Script

ReLU

$$\frac{df}{du} = 0 \qquad \frac{df}{du} = 1$$

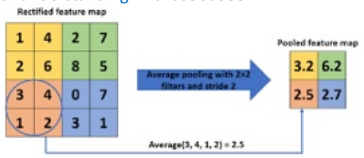


https://www.kaggle.com/code/dansbecker/rectified-linear-units-relu-in-deep-learning/notebook

Kirschner et al. (2022) *Nat Comm*

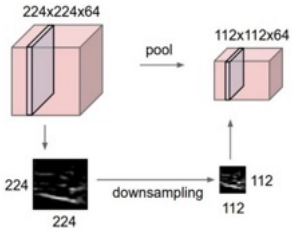
```
# 2. Convolutional Layers
x = Conv1D(256, kernel_size=2, activation='relu')(x)
x = AveragePooling1D(pool_size=2)(x)
x = BatchNormalization()(x)
```

https://towardsdatascience.com/batch-normalization-in-3-levels-of-understanding-14c2da90a338



Gholamalinezhad & Khosravi (2020) *arXiv*

CNN Script

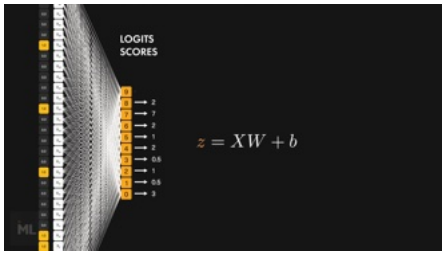


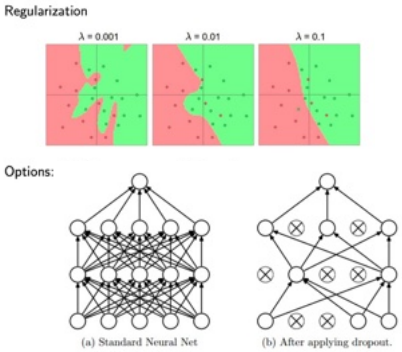
https://leonardoaraujosantos.gitbook.io/artificial-inteligence/machine_learning/deep_learning/pooling_layer

Kirschner et al. (2022) *Nat Comm*

CNN Script

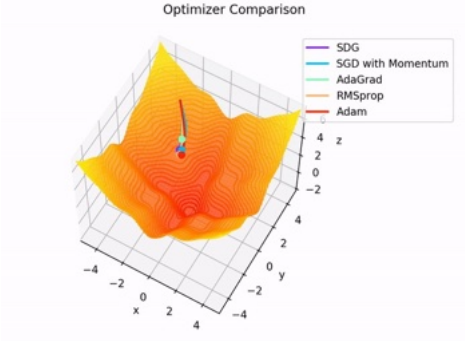
```
x = Flatten()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(num_classes, activation="softmax")(x)
```



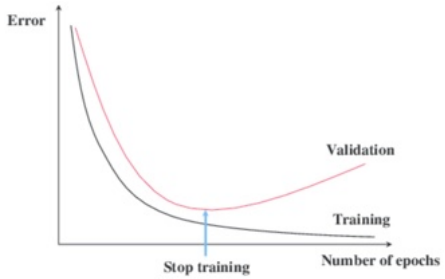


CNN Script

```
# Compile the CNN.
model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer='Adam',
              metrics=['accuracy'])

# We will use early stopping and save the model with the best val_accuracy.
earlyStopping = EarlyStopping(monitor='val_accuracy', patience=25, verbose=0, mode='max', restore_best_weights
### stop training when validation error increases (wait 25 epochs to see if there is any improvement).
```



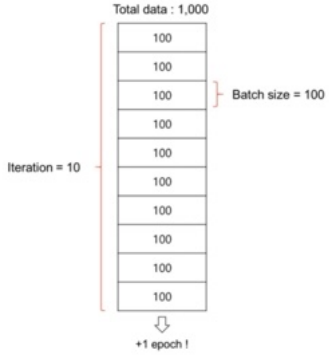
https://towardsdatascience.com/complete-guide-to-adam-optimization-1e5f29532c3d

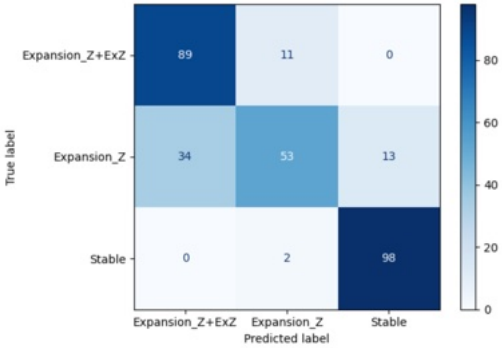


https://towardsdatascience.com/a-practical-introduction-to-early-stopping-in-machine-learning-550ac88bc8fd

CNN Script

```
#Run the CNN
history = model.fit(xtrain, ytrain, batch_size=batch_size,
          epochs=epochs,
          verbose=1,
          validation_data=(xval, yval),callbacks=[earlyStopping])
```





https://jerryan.medium.com/batch-size-a15958708a6